

Planificador Dieciochero

El Planificador Dieciochero

Tarea 1 — Procesos, Tuberías y Señales

Sistemas Operativos

Fecha de entrega: **29/09/2026** | Modalidad: **Grupos de máximo 2 personas**

1. Descripción

El señor Loyola es un hombre de tradiciones y ha tomado una decisión irrevocable: este año celebrará las Fiestas Patrias durante la semana completa, de inicio a fin, sin importar cuántos feriados apruebe el gobierno o si en el trabajo le dan permiso.

Pese a ser alguien con una reputación legendaria para la fiesta, el señor Loyola es una persona extremadamente organizada. Para él, la ramada no es un evento caótico, sino un sistema estructurado. Por ello, ha decidido construir, un **Simulador y Planificador de Actividades** para orquestar todas las combinaciones posibles de celebración.

2. Formato del Archivo

Cada día de celebración se modela matemáticamente como un Grafo Acíclico Dirigido (DAG). El simulador debe leer un archivo de texto plano (`plan.txt`) que describe las actividades, su duración y sus dependencias. En caso de que una actividad no tenga una duración, esta debe ser asignada de forma aleatoria en un rango entre 100 y 5000 milisegundos.

El formato de cada línea es el siguiente: `ID_Actividad : Nombre_Actividad : tiempo_ms : [Dependencia1, Dependencia2, ...]`

Ejemplo de `plan.txt`:

```
1 : prender_carbon : 500 :
2 : comprar_carne : 1200 :
3 : comprar_pan : 300 :
4 : asar_longaniza : 800 : 1, 2
5 : armar_choripan : 250 : 3, 4
6 : servir_mesa : 100 : 5
```

En este ejemplo, `asar_longaniza` (D) no puede comenzar hasta que el carbón esté prendido (A) y la carne comprada (B).

Detalle de los campos:

- **ID_Actividad:** Identificador único alfanumérico del nodo.
- **Nombre_Actividad:** Etiqueta descriptiva de la acción.
- **Tiempo (`tiempo_ms`):** El tiempo estimado de la actividad en **milisegundos**.
- **Dependencias:** Lista de IDs separados por comas. Estas actividades deben haber finalizado antes de que el nodo actual pueda ejecutarse.

3. Requisitos Funcionales

Su programa debe invocarse de la siguiente manera: `./planificador plan.txt K` (donde K es el límite de concurrencia).

3.1. Control de Concurrencia (K)

El planificador nunca debe tener más de K procesos creados, si hay más de K actividades, el sistema debe hacer que las tareas restantes esperen a que se liberen recursos.

3.2. Paso de Mensajes (Pipes)

Cuando una actividad finaliza su simulación, debe propagar un mensaje de texto acotado hacia su(s) actividad(es) dependiente(s) notificando su insumo.

3.3. Carga de trabajo

El simulador será sometido a pruebas de estrés cargando planificaciones de hasta **10000 actividades**.

4. Tolerancia a fallos e Inspecciones

4.1. Aislamiento de Errores

Si una actividad falla internamente, el simulador **no debe cerrarse abruptamente**. Debe abortar únicamente la rama del plan que dependía de esa actividad fallida.

4.2. Inspección de la Seremi (Ctrl+C)

Si el usuario presiona Ctrl+C (señal **SIGINT**), se simula la llegada de la autoridad. El planificador debe abortar todas las actividades.

5. Restricciones Técnicas

- Compilación en C (`gcc -Wall -Wextra -std=c17`) o C++ (`g++ -Wall -Wextra -std=c++17`).
- El código debe compilar en entornos Unix, en caso de recibir una tarea que no compile se evaluará con nota mínima.
- Prohibido usar hilos (*threads*) o mecanismos de sincronización de hilos.